

P1099R3

Using Enum

Published Proposal, 2019-01-20

This version:

<https://github.com/atomgalaxy/using-enum/using-enum.bs>

Authors:

Gašper Ažman <gasper.azman@gmail.com>

Jonathan Müller <jonathan.mueller@foonathan.net>

Audience:

EWG, CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Class enums are restricted namespaces. Let's extend the using declaration to them.

Table of Contents

1 Revision History

2 Motivation

3 Rationale

3.1 Consistency

3.2 Better Identifiers

3.3 Evidence of Need

4 Proposal

4.1 Syntax: `using enum IDENTIFIER`

4.2 Syntax: `using ENUM_ID::IDENTIFIER`

5 Examples

5.1 Strongly typed enums with global identifiers

5.2 Switching with no syntax overhead

6 Frequently Asked Questions

6.1 Has this been implemented?

6.2 Can I do this with unscoped enums?

6.3 Are you proposing mirroring the namespace alias syntax as well?

6.4 Why not allow `using enum struct/class ENUM_ID;`?

6.5 Why propose `using ENUM_ID::IDENTIFIER` at all?

7 Proposed Wording

7.1 Preface

7.2 Changes required to add `using enum IDENTIFIER;`

7.3 Changes required to add `using ENUM_ID::IDENTIFIER;`

8 Acknowledgements

References

Informative References

§ 1. Revision History

- r1: Typos. Changed target from LEWG to EWG.
- r2: Added FAQ and wording, and extended to regular enums.
- r3: Added examples. Nothing of substance changed.

§ 2. Motivation

The single biggest deterrent to use of scoped enumerations is the inability to associate them with a *using directive*.

— Dan Saks

Consider an enum class:

```
enum class rgba_color_channel { red, green, blue, alpha };
```

Currently, a switch using this enum looks as follows:

```
std::string_view to_string(rgba_color_channel channel) {  
    switch (channel) {  
        case rgba_color_channel::red:    return "red";  
        case rgba_color_channel::green:  return "green";  
        case rgba_color_channel::blue:   return "blue";  
        case rgba_color_channel::alpha:  return "alpha";  
    }  
}
```

The necessary repetition of the class enum name reduces legibility by introducing noise in contexts where the enum class is obvious.

To eliminate the noise penalty for introducing long (but descriptive) `enum class` names, this paper proposes that the statement

```
using enum rgba_color_channel;
```

introduce the enum member identifiers into the local scope so that they may be referred to unqualified.

Furthermore, the syntax

```
using rgba_color_channel::red;
```

should bring the identifier `red` into the local scope, so that it may be used unqualified.

The above example would then probably be written as

```

std::string_view to_string(rgba_color_channel channel) {
    switch (my_channel) {
        using enum rgba_color_channel;
        case red:    return "red";
        case green:  return "green";
        case blue:   return "blue";
        case alpha:  return "alpha";
    }
}

```

§ 3. Rationale

§ 3.1. Consistency

`enum class`es, as well as plain old `enums`, are not classes - they seem to be closer to namespaces consisting of `static constexpr` inline variables. The familiar `using` syntax that works for namespaces should therefore apply to them as well, in some fashion.

§ 3.2. Better Identifiers

The introduction of this feature would allow better naming of enums. Currently, enums are named with as short an identifier as possible, often to the point of absurdity, when they are reduced to completely nondescriptive abbreviations that only hint at their proper meaning. (Just what does `zfqc::add_op` really mean?)

With this feature, identifiers become available to unqualified lookup in local contexts where their source is obvious, giving control of lookup style back to the user of the enum, instead of baking name semantics into the type of the enum.

§ 3.3. Evidence of Need

At a casual search, we were able to locate this [thread on stackoverflow](#).

Anecdotally, 100% of people the authors have shown this to (~30) at CppCon have displayed a very enthusiastic response, with frequent comments of "I'd use enum classes but they are too verbose, this solves my problem!"

§ 4. Proposal

§ 4.1. Syntax: `using enum IDENTIFIER`

We propose the addition of a new `using enum` statement:

```
using enum IDENTIFIER;
```

The above statement introduces the members of the enumeration `IDENTIFIER` into the local namespace, thus enabling lookup without qualification. As usual, a name lookup ambiguity makes the program ill-formed (diagnostic required).

§ 4.2. Syntax: `using ENUM_ID::IDENTIFIER`

We propose to allow the syntax of

```
using ENUM_ID::IDENTIFIER
```

to introduce the `IDENTIFIER` into the local namespace, aliasing `ENUM_ID::IDENTIFIER`.

This would mirror the current syntax for introducing namespaced names into the current scope.

Note: this does not conflict with [\[P0945R0\]](#), because that paper only deals with the syntax `using name = id-expression`, which duplicates the enumerator name.

§ 5. Examples

§ 5.1. Strongly typed enums with global identifiers

This proposal lets you make strongly-typed enums still export their identifiers to namespace scope, therefore behaving like the old enums in that respect:

```

namespace my_lib {

enum class errcode {
    SUCCESS = 0,
    ENOMEM = 1,
    EAGAIN = 2,
    ET00SLOW = 3
};
using enum errcode;

}

namespace foo {

my_lib::errcode get_widget() {
    using namespace my_lib;
    return ET00SLOW; // works, and conversions to int don't.
}

}

```

§ 5.2. Switching with no syntax overhead

The proposal allows for importing enums inside the switch body, which is a scope, and using them for labels:

```

enum class rgba_color_channel { red, green, blue, alpha};

std::string_view to_string(rgba_color_channel channel) {
    switch (my_channel) {
        using enum rgba_color_channel;
        case red:    return "red";
        case green:  return "green";
        case blue:   return "blue";
        case alpha:  return "alpha";
    }
}

```

§ 6. Frequently Asked Questions

§ 6.1. Has this been implemented?

Yes. The author has an implementation in clang. It has not been reviewed or released yet, however. There do not seem to be major issues with implementation.

§ 6.2. Can I do this with unscoped enums?

Yes. The motivation for that is the pattern

```
class foo {  
    enum bar {  
        A,  
        B,  
        C  
    };  
};
```

which was superseded by scoped enums. With the feature this paper proposes one can bring `A`, `B` and `C` into the local scope by invoking:

```
using enum foo::bar;
```

§ 6.3. Are you proposing mirroring the namespace alias syntax as well?

No. We already have a way to do that, and it looks like this:

```
using my_alias = my::name_space::enum_name;
```

In addition, [\[P0945R0\]](#) proposes deprecating namespace aliases in favor of generalized `using name = id_expression`, so doing this would go counter the current movement of the standard.

§ 6.4. Why not allow `using enum struct/class ENUM_ID`?

Because would have been a needless complication and would introduce another layer of "`struct` and `class` don't match" linter errors that current `classes` and `structs` already have with forward declarations.

§ 6.5. Why propose `using ENUM_ID::IDENTIFIER` at all?

... given that the following already works:

```
constexpr auto red = rgba_color_channel::red;
```

and that, given [\[P0945R0\]](#), this will work:

```
using red = rgba_color_channel::red;
```

The reason is "DRY" - don't repeat yourself - one is forced to repeat the name of the enumerator. That said, the authors are perfectly willing to throw this part of the paper out if the `using enum ENUM_ID` piece gets consensus and this is the stumbling block.

§ 7. Proposed Wording

§ 7.1. Preface

The authors are new at this, and welcome suggestions for wording.

The intention is to effectively mirror the wording for namespaces from [\[namespace.udir\]](#), and pare it down to only things that apply to enumerators.

All wording is relative to the working draft of the ISO/IEC IS 14882: N4765.

§ 7.2. Changes required to add `using enum IDENTIFIER`;

In chapter [\[dcl.dcl\]](#), in [\[dcl.enum\]](#), add section titled "Using Directive", with the stable reference "[\[enum.udir\]](#)".

Add:

using-enum-directive:

*attribute-specifier-seq*_{opt} *using enum nested-name-specifier*_{opt} *enum-name*

1. A *using-enum-directive* shall not appear in class scope, but may appear in namespace scope or in block scope. The optional *attribute-specifier-seq* appertains to the *using-enum-directive*. The *enum-name* in a *using-enum-directive* shall name a scoped enumeration or an unscoped enumeration.
2. A *using-enum-directive* specifies that the *enumerators* in the nominated scoped enumeration or unscoped enumeration can be used in the scope in which the *using-enum-directive* appears after the *using-enum-directive*. During unqualified name lookup ([basic.lookup.unqual]), the names appear as if they were declared in the scope where the *using-enum-directive* appears.
3. A *using-enum-directive* does not add any members to the declarative region in which it appears. [

Example:

```
namespace A {
    enum class e { i = 0 };
    using enum A::e; // make e::i visible in A
    namespace B {
        enum class f { i = 1 };
        using enum A::B::f;
        void f1() {
            i;          // OK, A::B::f::i visible in B and hides A::e::i
        }
    }
}
namespace C {
    using enum A::e;
    using enum A::B::f;
    void f2() {
        i;          // ambiguous, A::e::i or A::B::f::i?
    }
}
```

— end example]

Under [basic.def], add (just after *using-directive*) (and renumber section):

2.17. — it is a *using-enum-directive*

§ 7.3. Changes required to add `using ENUM_ID::IDENTIFIER;`

In chapter [namespace.udecl], remove:

~~7. A *using-declaration* shall not name a scoped enumerator.~~

Then add:

7. A *using-declaration* that names a scoped or unscoped enumerator shall become a synonym for that enumerator in the *using-declaration*'s declarative region. [*Note*: this is equivalent to declaring that enumerator in the *using-declaration*'s declarative region. — *end note*]

§ 8. Acknowledgements

The authors would like to thank Marcel Ebmer and Lisa Lippincott for early feedback, and the members of the BSI C++ WG for further feedback, especially Graham Haynes and Barry Revzin. Even further feedback was provided by Tomas Puerle, who encouraged us to extend it to `enum`s, and Dan Saks for the permission to include a quotation from him.

§ References

§ Informative References

[P0945R0]

Richard Smith. [Generalizing alias declarations](https://wg21.link/p0945r0). 10 February 2018. URL: <https://wg21.link/p0945r0>